

Proposal: Conversational RAG-Powered E-commerce Ecosystem

Team Members: Saad Shakeel, Ashhad Hassan, Muhammad Bilal | **Team Leader:** [Saad Shakeel]

Advisor: [Engr Abdul Rehman] | **Date:** February 16, 2026

1. Overview

The motivation for this project is to eliminate the "vocabulary gap" in traditional e-commerce, where users struggle with rigid filters and keyword-based searches. We are delivering a new AI-first product for online retailers that replaces the search bar with a Retrieval-Augmented Generation (RAG) chatbot. This agent acts as a virtual consultant, understanding complex feature requests in natural language.

2. Project Detail

2.1 Problem or Challenge

Online shoppers in our region frequently experience "filter fatigue," spending more time toggling checkboxes than evaluating products. Traditional keyword search fails to understand intent (e.g., "something for a formal outdoor wedding in hot weather"), leading to high bounce rates for local digital storefronts.

2.2 Project Vision

Our vision is to humanize digital commerce by providing a "natural language first" interface. We aim to ensure that no user is ever more than one sentence away from finding the exact product that meets their nuanced, multi-feature requirements.

2.3 Project Feasibility Study

- **2.3.1 Technical Feasibility:** We will use LangChain for RAG orchestration and a Vector Database (Pinecone/ChromaDB) to store product embeddings. These are mature, documented technologies.
- **2.3.2 Operational Feasibility:** The system requires no training for end-users, as it mimics standard human conversation.
- **2.3.3 Economic Feasibility:** Increased conversion rates and reduced reliance on manual customer support make this a high-ROI investment.

2.4 Project Goals

- **Functional:** Implement a multi-turn RAG agent grounded in a product database.
- **Technological:** Achieve sub-second retrieval from a high-dimensional vector space.

- **Quality:** Zero hallucinations; all responses must be verified by the product catalog.
- **Other:** High mobile responsiveness for conversational interfaces.

2.5 Project Abstract

This project introduces a "Consultative E-commerce" platform that moves away from traditional database queries. By utilizing RAG, the system retrieves product metadata—such as materials, dimensions, and user reviews—and passes them to an LLM to generate personalized advice. Users can specify complex combinations of features, and the system will explain *why* specific items were selected.

2.9 Proposed Solution & Architecture

Our architecture uses a **Layered RAG Approach**:

1. **Frontend:** React-based chat interface.
2. **Logic Layer:** FastAPI handling the conversational state.
3. **AI Layer:** A RAG pipeline connecting the LLM to a Vector Store containing product embeddings.
4. **Data Layer:** PostgreSQL for transactions and Pinecone for semantic search.

2.11 Feature List (20 Core Domain Features)

In accordance with the template, these focus on core domain functionality rather than generic features like login/logout.

1. **Natural Language Attribute Filtering:** Mapping words like "cheap" or "large" to database columns.
2. **Multi-Constraint Search:** Filtering by price, color, and size in a single text input.
3. **Dynamic Comparison Grid:** Comparing two selected items side-by-side.
4. **Price Range Recognition:** Automatically setting SQL BETWEEN filters from chat text.
5. **Tag-Based Discovery:** Finding items via hidden tags (e.g., "professional," "waterproof").
6. **Weighted Sorting:** Prioritizing search results based on user-specified "must-have" features.
7. **Conversational Size Selection:** Choosing product variants via chat input.
8. **Real-Time Stock Check:** Querying the inventory table via the chat interface.
9. **Related Item Cross-Selling:** Logic-based suggestions (e.g., "Users also bought this charger").
10. **Feature-Based Product Bundling:** Compiling sets of items that meet a specific user intent.
11. **JSONB Detail Extraction:** Displaying complex specs from PostgreSQL JSONB columns.

12. **Text-Based FAQ Retrieval:** Matching user questions to a store policy database table.
13. **Guided Follow-up Prompts:** Backend-generated questions when a search is too broad.
14. **Cart Feature Summary:** Showing a text summary of all items and their key specs in the cart.
15. **Promo Code Validation:** Processing text-based discount codes via FastAPI.
16. **Compatibility Logic:** Checking if two items (e.g., phone and case) share the same model tag.
17. **Popularity Filtering:** Surfacing items with high ratings/sales through chat.
18. **Review Search:** Scanning user review text for specific feature mentions.
19. **Command-Based Cart Management:** "Add the blue one to my cart" text processing.
20. **Order Tracking Query:** Checking order_status in the database using an order ID via chat.

2.12 Software Development Methodology

We will follow the **Agile Scrum Methodology**. This allows for iterative "Prompt Engineering" and "Embedding Optimization" cycles, ensuring the AI model improves with every 2-week sprint.

2.13 Proposed Infrastructure Platform and Tools

- **Backend:** Python, FastAPI.
- **Frontend:** Next.js, Tailwind CSS.
- **Database:** PostgreSQL (Cloud Managed).

2.16 References

1. P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *NeurIPS*, 2020.
2. S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Found. Trends Inf. Retr.*, 2009.